

Laboratorio IPsec IKEv2/ESP

Túnel IPsec con strongSwan sobre Docker

Autor:	Diego Medardo Saavedra García
Programa:	Máster en Ciberseguridad
Universidad:	Universidad Complutense de Madrid
Fecha:	Junio 2026
Repositorio:	github.com/statick88/ipsec-lab
Docker Hub:	hub.docker.com/r/statick/ipsec-lab

Contenido

1. Resumen Ejecutivo
2. Objetivos
3. Arquitectura del Laboratorio
4. Metodología de Implementación
5. Configuración y Algoritmos
6. Resultados y Evidencias (CLI real)
7. Validación con Wireshark
8. Automatización y CI/CD
9. Conclusiones
10. Referencias

1. Resumen Ejecutivo

Este informe documenta el diseño, implementación y validación de un laboratorio de seguridad IPsec que demuestra el funcionamiento de túneles IKEv2 con encapsulación ESP entre dos gateways simulados mediante network namespaces en Docker.

El laboratorio utiliza **strongSwan 5.9.5** sobre **Ubuntu 22.04**, con algoritmos AES-CBC-256 para confidencialidad y HMAC-SHA-256-128 para integridad. La autenticación se realiza mediante Pre-Shared Key (PSK).

Todos los resultados mostrados en este informe son **salidas reales de CLI** capturadas durante la ejecución del laboratorio, no simulaciones ni gráficas generadas.

Componente	Estado	Detalle
IKEv2 SA	ESTABLISHED	AES_CBC_256 / HMAC_SHA2_256_128
CHILD SA (ESP)	INSTALLED	Modo tunnel
Tráfico cifrado	Verificado	ICMP ping entre subnets
Captura pcap	Generada	ESP + ICMP
Wireshark keys	Exportadas	Formato CSV
Tests	13/13	Unit + E2E + Security
CI/CD	Funcional	GitHub Actions
Docker Hub	Publicada	statick/ipsec-lab:latest

2. Objetivos

Objetivo General

Demostrar el funcionamiento de un túnel IPsec IKEv2 con encapsulación ESP en un entorno controlado, validando los algoritmos de cifrado, integridad y autenticación.

Objetivos Específicos

- Configurar dos gateways IPsec (east/west) utilizando network namespaces aislados
- Establecer un túnel IKEv2 con autenticación PSK y algoritmos AES-CBC-256/HMAC-SHA-256
- Validar el encapsulamiento ESP en tráfico entre subnets protegidos
- Capturar y decodificar paquetes ESP utilizando Wireshark/tshark
- Generar evidencia académica completa con salidas reales de CLI
- Automatizar el entorno mediante Docker y Makefile
- Implementar pipeline CI/CD con tests automatizados y escaneo de seguridad

3. Arquitectura del Laboratorio

El laboratorio se ejecuta en un único contenedor Docker con privilegios, utilizando network namespaces para simular dos gateways IPsec independientes conectados por un par veth.

```
$ ip netns list
ns-east (id: 3)
ns-west (id: 4)

$ ip netns exec ns-east ip addr show veth-east
23: veth-east@if22: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen
    link/ether da:5b:9c:d4:65:d6 brd ff:ff:ff:ff:ff:ff link-netns ns-west
    inet 10.0.0.1/30 scope global veth-east
        valid_lft forever preferred_lft forever
    inet6 fe80::d85b:9cff:fed4:65d6/64 scope link
        valid_lft forever preferred_lft forever

$ ip netns exec ns-west ip addr show veth-west
22: veth-west@if23: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen
    link/ether 56:c6:1f:0d:e5:ca brd ff:ff:ff:ff:ff:ff link-netns ns-east
    inet 10.0.0.2/30 scope global veth-west
        valid_lft forever preferred_lft forever
    inet6 fe80::54c6:1fff:fe0d:e5ca/64 scope link
        valid_lft forever preferred_lft forever

$ ip netns exec ns-east ip addr show lo
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet 10.10.0.1/24 scope global lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever

$ ip netns exec ns-west ip addr show lo
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet 10.20.0.1/24 scope global lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
```

Figura 1. Topología de red real: namespaces, interfaces y direccionamiento

Elemento	Configuración	Propósito
Namespace East	ns-east	Gateway initiator (10.0.0.1/30)
Namespace West	ns-west	Gateway responder (10.0.0.2/30)
Veth Pair	veth-east ↔ veth-west	Enlace tránsito 10.0.0.0/30
Subnet East	10.10.0.1/24 (lo)	Red protegida este
Subnet West	10.20.0.1/24 (lo)	Red protegida oeste
Software	strongSwan 5.9.5 (apt)	Daemon IPsec IKEv2

SO Base	Ubuntu 22.04	Contenedor Docker
---------	--------------	-------------------

4. Metodología de Implementación

Fase 1: Diseño del Entorno

Se seleccionó Docker + Colima (VZ/VirtioFS) como plataforma. Los network namespaces proporcionan aislamiento de red sin múltiples contenedores, y strongSwan 5.9.5 desde apt incluye todos los plugins necesarios.

Fase 2: Configuración de Red

- Creación de namespaces: `ip netns add ns-east/ns-west`
- Par veth: `veth-east ↔ veth-west`
- Direccionamiento tránsito: `10.0.0.1/30 ↔ 10.0.0.2/30`
- Subnets protegidos: `10.10.0.0/24` (lo ns-east) y `10.20.0.0/24` (lo ns-west)

Fase 3: Configuración IPsec

Configuración stroke (`ipsec.conf` + `ipsec.secrets`). Cada gateway ejecuta su propia instancia de charon con archivos PID y socket separados.

Fase 4: Validación

- Verificación SA: `ipsec statusall`
- Tráfico cifrado: ping entre subnets con monitoreo XFRM
- Captura: `tcpdump` en veth-east
- Claves: parsing de `ip xfrm state` a CSV
- Decodificación: verificación con `tshark`

5. Configuración y Algoritmos

Propuesta IKE (Phase 1)

Parámetro	Valor	RFC
Cipher	AES_CBC_256	RFC 3602
Integrity	HMAC_SHA2_256_128	RFC 4868
PRF	PRF_HMAC_SHA2_256	RFC 7296
DH Group	MODP_2048 (Group 14)	RFC 3526
Lifetime	28800 s	Default

Propuesta ESP (Phase 2)

Parámetro	Valor	RFC
Protocol	ESP (IP Protocol 50)	RFC 4303
Cipher	AES_CBC_256	RFC 3602
Integrity	HMAC_SHA2_256_128	RFC 4868
Mode	Tunnel	RFC 4303
Lifetime	3600 s	Default

Configuración IPsec (ipsec.conf real)

```
$ cat /etc/ipsec.conf
# strongSwan IPsec configuration file for gw-east (initiator)
# Stroke-based configuration for traditional strongSwan

config setup
    # uniqueids=yes ensures each child SA gets its own ID
    uniqueids=yes

conn %default
    # Authentication method: Pre-Shared Key
    authby=secret
    # IKEv2 key exchange
    keyexchange=ikev2
    # ESP encryption algorithm
    esp=aes256-sha256-modp2048!
    # IKE encryption algorithm
    ike=aes256-sha256-modp2048!
    # Lifetime for IKE SA (8 hours)
    ikelifetime=28800s
    # Lifetime for ESP SA (1 hour)
    lifetime=3600s
    # Margin for rekeying (10 minutes)
    rekeymargin=600s
```

```

# Margintime (5 minutes)
margintime=300s
# DPDDelay: Dead Peer Detection delay
dpddelay=30s
# DPDTimeout: Dead Peer Detection timeout
dpdtimeout=150s
# DPDAction: restart on DPD failure
dpdaction=restart
# Auto=add means load but don't start
auto=add

# Connection between gw-east and gw-west
conn gw-east-to-gw-west
# Left side (initiator)
left=10.0.0.1
leftsubnet=10.10.0.0/24
leftfirewall=yes
# Right side (responder)
right=10.0.0.2
rightsubnet=10.20.0.0/24
# Start connection automatically
auto=start

```

Figura 2. Archivo ipsec.conf real del gateway east (initiator)

Pre-Shared Key

```

$ cat /etc/ipsec.secrets
# strongSwan IPsec secrets file for gw-east (initiator)
# Pre-Shared Key for IKEv2 authentication

# PSK between gw-east and gw-west
10.0.0.1 10.0.0.2 : PSK "StrongSwanLab2024!"

```

Figura 3. Archivo ipsec.secrets con PSK

6. Resultados y Evidencias

Las siguientes salidas son **comandos reales ejecutados durante el laboratorio**. Cada bloque muestra el prompt, el comando y la salida exacta del sistema.

6.1 Enrutamiento entre Namespaces

```
$ ip netns exec ns-east ip route
10.0.0.0/30 dev veth-east proto kernel scope link src 10.0.0.1
10.20.0.0/24 via 10.0.0.2 dev veth-east

$ ip netns exec ns-west ip route
10.0.0.0/30 dev veth-west proto kernel scope link src 10.0.0.2
10.10.0.0/24 via 10.0.0.1 dev veth-west
```

Figura 4. Tablas de enrutamiento de ns-east y ns-west

6.2 Estado IPsec (IKE SA + CHILD SA)

El comando `ipsec statusall` muestra las SA establecidas: IKE SA en estado **ESTABLISHED** y CHILD SA (ESP) en estado **INSTALLED**.

```
$ ip netns exec ns-east ipsec statusall
Status of IKE charon daemon (strongSwan 5.9.5, Linux 6.8.0-117-generic, aarch64):
  uptime: 81 seconds, since Jun 20 09:38:54 2026
  malloc: sbrk 2682880, mmap 0, used 590640, free 2092240
  worker threads: 11 of 16 idle, 5/0/0/0 working, job queue: 0/0/0/0, scheduled: 3
  loaded plugins: charon aes rc2 sha2 sha1 md5 mgf1 random nonce x509 revocation constraints pubkey pkcs1 pkcs7
Listening IP addresses:
  10.0.0.1
Connections:
gw-east-to-gw-west: 10.0.0.1...10.0.0.2 IKEv2, dpddelay=30s
gw-east-to-gw-west: local: [10.0.0.1] uses pre-shared key authentication
gw-east-to-gw-west: remote: [10.0.0.2] uses pre-shared key authentication
gw-east-to-gw-west: child: 10.10.0.0/24 === 10.20.0.0/24 TUNNEL, dpdaction=restart
Security Associations (1 up, 0 connecting):
gw-east-to-gw-west[1]: ESTABLISHED 81 seconds ago, 10.0.0.1[10.0.0.1]...10.0.0.2[10.0.0.2]
gw-east-to-gw-west[1]: IKEv2 SPIs: 769f6350673feff9_i* 5e7f9a506b7c2637_r, pre-shared key reauthentication in 7
gw-east-to-gw-west[1]: IKE proposal: AES_CBC_256/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/MODP_2048
gw-east-to-gw-west{1}: INSTALLED, TUNNEL, reqid 1, ESP SPIs: ccaad0a3_i ceecf4cf_o
gw-east-to-gw-west{1}: AES_CBC_256/HMAC_SHA2_256_128, 1260 bytes_i (15 pkts, 37s ago), 1260 bytes_o (15 pkts,
gw-east-to-gw-west{1}: 10.10.0.0/24 === 10.20.0.0/24

$ ip netns exec ns-east ipsec status
Security Associations (1 up, 0 connecting):
gw-east-to-gw-west[1]: ESTABLISHED 81 seconds ago, 10.0.0.1[10.0.0.1]...10.0.0.2[10.0.0.2]
gw-east-to-gw-west{1}: INSTALLED, TUNNEL, reqid 1, ESP SPIs: ccaad0a3_i ceecf4cf_o
gw-east-to-gw-west{1}: 10.10.0.0/24 === 10.20.0.0/24
```

Figura 5. Estado IPsec: SA IKEv2 + CHILD SA (ESP) instaladas

6.3 Estado XFRM (IPsec Kernel)

El estado XFRM del kernel muestra las SA activas con los algoritmos reales: AES-CBC-256 (cipher) y HMAC-SHA-256-128 (auth). Los contadores de bytes y paquetes confirman el tráfico cifrado.

```
$ ip netns exec ns-east ip xfrm state
```

```

src 10.0.0.1 dst 10.0.0.2
■proto esp spi 0xceecf4cf reqid 1 mode tunnel
■replay-window 0 flag af-unspec
■auth-trunc hmac(sha256) 0x358148d89914f18fe880f9c019b610ff774d8d732651b66801ef12878d98d772 128
■enc cbc(aes) 0x54b1e312e4e44a81786752a12e4f57cf7f6ee6a8ad81ef28264876b2b4787855
■lastused 2026-06-20 09:39:37
■anti-replay context: seq 0x0, oseq 0xf, bitmap 0x00000000
src 10.0.0.2 dst 10.0.0.1
■proto esp spi 0xccaad0a3 reqid 1 mode tunnel
■replay-window 32 flag af-unspec
■auth-trunc hmac(sha256) 0xba8d650b0e3cb60cd20c0ccda315734e363a564e9e742b0dfba89b89023f449b 128
■enc cbc(aes) 0xa229a6eb9a6962fc42a2eb09b43c3a62e610ec1caf153daf221eae0a5b9185a2
■lastused 2026-06-20 09:39:37
■anti-replay context: seq 0xf, oseq 0x0, bitmap 0x00007fff

$ ip netns exec ns-east ip xfrm policy
src 10.10.0.0/24 dst 10.20.0.0/24
■dir out priority 375423
■tmpl src 10.0.0.1 dst 10.0.0.2
■proto esp spi 0xceecf4cf reqid 1 mode tunnel
src 10.20.0.0/24 dst 10.10.0.0/24
■dir fwd priority 375423
■tmpl src 10.0.0.2 dst 10.0.0.1
■proto esp reqid 1 mode tunnel
src 10.20.0.0/24 dst 10.10.0.0/24
■dir in priority 375423
■tmpl src 10.0.0.2 dst 10.0.0.1
■proto esp reqid 1 mode tunnel
src 0.0.0.0/0 dst 0.0.0.0/0
■socket in priority 0
src 0.0.0.0/0 dst 0.0.0.0/0
■socket out priority 0
src 0.0.0.0/0 dst 0.0.0.0/0
■socket in priority 0
src 0.0.0.0/0 dst 0.0.0.0/0
■socket out priority 0
src ::/0 dst ::/0
■socket in priority 0
src ::/0 dst ::/0
■socket out priority 0
src ::/0 dst ::/0
■socket in priority 0
src ::/0 dst ::/0
■socket out priority 0

$ ip netns exec ns-west ip xfrm state
src 10.0.0.2 dst 10.0.0.1
■proto esp spi 0xccaad0a3 reqid 1 mode tunnel
■replay-window 0 flag af-unspec
■auth-trunc hmac(sha256) 0xba8d650b0e3cb60cd20c0ccda315734e363a564e9e742b0dfba89b89023f449b 128
■enc cbc(aes) 0xa229a6eb9a6962fc42a2eb09b43c3a62e610ec1caf153daf221eae0a5b9185a2
■lastused 2026-06-20 09:39:37
■anti-replay context: seq 0x0, oseq 0xf, bitmap 0x00000000
src 10.0.0.1 dst 10.0.0.2
■proto esp spi 0xceecf4cf reqid 1 mode tunnel
■replay-window 32 flag af-unspec
■auth-trunc hmac(sha256) 0x358148d89914f18fe880f9c019b610ff774d8d732651b66801ef12878d98d772 128
■enc cbc(aes) 0x54b1e312e4e44a81786752a12e4f57cf7f6ee6a8ad81ef28264876b2b4787855
■lastused 2026-06-20 09:39:37
... (1 líneas más)

```

Figura 6. Estado XFRM: SA ESP con algoritmos y claves reales

6.4 Prueba de Conectividad (Ping Cifrado)

El ping entre subnets protegidos (10.10.0.1 → 10.20.0.1) demuestra que el tráfico se encapsula correctamente mediante ESP a través del túnel.

```
$ ip netns exec ns-east ping -c 5 10.20.0.1
PING 10.20.0.1 (10.20.0.1) 56(84) bytes of data.
64 bytes from 10.20.0.1: icmp_seq=1 ttl=64 time=0.066 ms
64 bytes from 10.20.0.1: icmp_seq=2 ttl=64 time=0.209 ms
64 bytes from 10.20.0.1: icmp_seq=3 ttl=64 time=0.200 ms
64 bytes from 10.20.0.1: icmp_seq=4 ttl=64 time=0.259 ms
64 bytes from 10.20.0.1: icmp_seq=5 ttl=64 time=0.322 ms

--- 10.20.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4008ms
rtt min/avg/max/mdev = 0.066/0.211/0.322/0.084 ms
```

Figura 7. Ping cifrado: 10.10.0.1 → 10.20.0.1 a través del túnel ESP

6.5 Estado IPsec Después del Tráfico

Después de generar tráfico, los contadores de bytes incrementan confirmando que el cifrado ESP está operativo.

```
$ ip netns exec ns-east ipsec statusall
Status of IKE charon daemon (strongSwan 5.9.5, Linux 6.8.0-117-generic, aarch64):
  uptime: 85 seconds, since Jun 20 09:38:54 2026
  malloc: sbrk 2682880, mmap 0, used 594256, free 2088624
  worker threads: 11 of 16 idle, 5/0/0/0 working, job queue: 0/0/0/0, scheduled: 3
  loaded plugins: charon aes rc2 sha2 sha1 md5 mgf1 random nonce x509 revocation constraints pubkey pkcs1 pkcs7
Listening IP addresses:
  10.0.0.1
Connections:
gw-east-to-gw-west: 10.0.0.1...10.0.0.2 IKEv2, dpddelay=30s
gw-east-to-gw-west:  local:  [10.0.0.1] uses pre-shared key authentication
gw-east-to-gw-west: remote: [10.0.0.2] uses pre-shared key authentication
gw-east-to-gw-west:  child:  10.10.0.0/24 === 10.20.0.0/24 TUNNEL, dpdaction=restart
Security Associations (1 up, 0 connecting):
gw-east-to-gw-west[1]: ESTABLISHED 85 seconds ago, 10.0.0.1[10.0.0.1]...10.0.0.2[10.0.0.2]
gw-east-to-gw-west[1]: IKEv2 SPIs: 769f6350673feff9_i* 5e7f9a506b7c2637_r, pre-shared key reauthentication in 7
gw-east-to-gw-west[1]: IKE proposal: AES_CBC_256/HMAC_SHA2_256_128/PRF_HMAC_SHA2_256/MODP_2048
gw-east-to-gw-west{1}:  INSTALLED, TUNNEL, reqid 1, ESP SPIs: ccaad0a3_i ceecf4cf_o
gw-east-to-gw-west{1}:  AES_CBC_256/HMAC_SHA2_256_128, 1680 bytes_i (20 pkts, 0s ago), 1680 bytes_o (20 pkts, 0
gw-east-to-gw-west{1}:  10.10.0.0/24 === 10.20.0.0/24
```

Figura 8. Estado IPsec después de generar tráfico cifrado

6.6 Claves ESP para Wireshark

El archivo esp_sa se genera parseando ip xfrm state. Formato CSV compatible con Wireshark para descifrado offline.

```
$ ip netns exec ns-east ip xfrm state
src 10.0.0.1 dst 10.0.0.2
■proto esp spi 0xceecf4cf reqid 1 mode tunnel
■replay-window 0 flag af-unspec
■auth-trunc hmac(sha256) 0x358148d89914f18fe880f9c019b610ff774d8d732651b66801ef12878d98d772 128
■enc cbc(aes) 0x54b1e312e4e44a81786752a12e4f57cf7f6ee6a8ad81ef28264876b2b4787855
■lastused 2026-06-20 09:40:19
```

```

■anti-replay context: seq 0x0, oseq 0x14, bitmap 0x00000000
src 10.0.0.2 dst 10.0.0.1
■proto esp spi 0xccaad0a3 reqid 1 mode tunnel
■replay-window 32 flag af-unspec
■auth-trunc hmac(sha256) 0xba8d650b0e3cb60cd20c0ccda315734e363a564e9e742b0dfba89b89023f449b 128
■enc cbc(aes) 0xa229a6eb9a6962fc42a2eb09b43c3a62e610ec1caf153daf221eae0a5b9185a2
■lastused 2026-06-20 09:40:19
■anti-replay context: seq 0x14, oseq 0x0, bitmap 0x000fffff

=== esp_sa file (Wireshark CSV format) ===
"IPv4","10.0.0.1","10.0.0.2","0xceecf4cf","AES-CBC [RFC3602]","0x54b1e312e4e44a81786752a12e4f57cf7f6ee6a8ad81ef"
"IPv4","10.0.0.2","10.0.0.1","0xccaad0a3","AES-CBC [RFC3602]","0xa229a6eb9a6962fc42a2eb09b43c3a62e610ec1caf153d"
"IPv4","10.0.0.2","10.0.0.1","0xccaad0a3","AES-CBC [RFC3602]","0xa229a6eb9a6962fc42a2eb09b43c3a62e610ec1caf153d"
"IPv4","10.0.0.1","10.0.0.2","0xceecf4cf","AES-CBC [RFC3602]","0x54b1e312e4e44a81786752a12e4f57cf7f6ee6a8ad81ef"

```

Figura 9. Claves ESP: estado XFRM + formato CSV para Wireshark

6.7 Análisis de Captura de Paquetes

La captura capture.pcap contiene paquetes ESP y ICMP decodificado. El análisis con tcpdump confirma que el tráfico entre gateways se cifra con ESP.

```
$ tcpdump -r /wireshark_keys/capture.pcap -nn
reading from file /wireshark_keys/capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
09:39:07.909115 IP 10.0.0.1 > 10.0.0.2: ESP(spi=0xceecf4cf,seq=0x1), length 136
09:39:07.909171 IP 10.0.0.2 > 10.0.0.1: ESP(spi=0xccaad0a3,seq=0x1), length 136
09:39:07.909171 IP 10.20.0.1 > 10.10.0.1: ICMP echo reply, id 47018, seq 1, length 64
09:39:08.939669 IP 10.0.0.1 > 10.0.0.2: ESP(spi=0xceecf4cf,seq=0x2), length 136
09:39:08.939722 IP 10.0.0.2 > 10.0.0.1: ESP(spi=0xccaad0a3,seq=0x2), length 136
09:39:08.939722 IP 10.20.0.1 > 10.10.0.1: ICMP echo reply, id 47018, seq 2, length 64
09:39:09.963614 IP 10.0.0.1 > 10.0.0.2: ESP(spi=0xceecf4cf,seq=0x3), length 136
09:39:09.963675 IP 10.0.0.2 > 10.0.0.1: ESP(spi=0xccaad0a3,seq=0x3), length 136
09:39:09.963675 IP 10.20.0.1 > 10.10.0.1: ICMP echo reply, id 47018, seq 3, length 64
09:39:10.983017 IP 10.0.0.1 > 10.0.0.2: ESP(spi=0xceecf4cf,seq=0x4), length 136
09:39:10.983077 IP 10.0.0.2 > 10.0.0.1: ESP(spi=0xccaad0a3,seq=0x4), length 136
09:39:10.983077 IP 10.20.0.1 > 10.10.0.1: ICMP echo reply, id 47018, seq 4, length 64
09:39:12.011387 IP 10.0.0.1 > 10.0.0.2: ESP(spi=0xceecf4cf,seq=0x5), length 136
09:39:12.011441 IP 10.0.0.2 > 10.0.0.1: ESP(spi=0xccaad0a3,seq=0x5), length 136
09:39:12.011441 IP 10.20.0.1 > 10.10.0.1: ICMP echo reply, id 47018, seq 5, length 64
09:39:12.852285 IP 10.0.0.1 > 10.0.0.2: ESP(spi=0xceecf4cf,seq=0x6), length 136
09:39:12.852383 IP 10.0.0.2 > 10.0.0.1: ESP(spi=0xccaad0a3,seq=0x6), length 136
09:39:12.852383 IP 10.20.0.1 > 10.10.0.1: ICMP echo reply, id 47018, seq 6, length 64
09:39:13.878242 IP 10.0.0.1 > 10.0.0.2: ESP(spi=0xceecf4cf,seq=0x7), length 136
09:39:13.878300 IP 10.0.0.2 > 10.0.0.1: ESP(spi=0xccaad0a3,seq=0x7), length 136
09:39:13.878300 IP 10.20.0.1 > 10.10.0.1: ICMP echo reply, id 47018, seq 7, length 64
09:39:14.904856 IP 10.0.0.1 > 10.0.0.2: ESP(spi=0xceecf4cf,seq=0x8), length 136
09:39:14.904916 IP 10.0.0.2 > 10.0.0.1: ESP(spi=0xccaad0a3,seq=0x8), length 136
09:39:14.904916 IP 10.20.0.1 > 10.10.0.1: ICMP echo reply, id 47018, seq 8, length 64

=== Packet count ===
Total packets: 25
ESP packets: 16
ICMP packets: 8
```

Figura 10. Análisis de captura: paquetes ESP + ICMP decodificados

7. Validación con Wireshark

Para validar el tráfico cifrado a nivel de paquete, se exportan las claves ESP del estado XFRM a un archivo compatible con Wireshark.

Proceso de Decodificación

1. **Exportación de claves:** `run-lab.sh parsea ip xfrm state → esp_sa`
2. **Formato CSV:** IPv4, src, dst, SPI, algoritmo_cifrado, clave_cifrado, algoritmo_integridad, clave_integridad
3. **Configuración Wireshark:** Preferences → Protocols → ESP → Enable decryption
4. **Verificación tshark:** `tshark -r capture.pcap -o "esp.enable_encryption_decode: TRUE"`

Resultado: Paquetes Decodificados

La decodificación muestra paquetes ESP conteniendo ICMP (ping), confirmando que el tráfico entre subnets protegidos se encapsula correctamente.

#	Origen	Destino	Protocolo	Detalle
1	10.0.0.1	10.0.0.2	ESP	SPI=0xceecf4cf, seq=0x1
2	10.0.0.2	10.0.0.1	ESP	SPI=0xccaad0a3, seq=0x1
3	10.20.0.1	10.10.0.1	ICMP	Echo reply (decoded from ESP)
4	10.0.0.1	10.0.0.2	ESP	SPI=0xceecf4cf, seq=0x2
5	10.0.0.2	10.0.0.1	ESP	SPI=0xccaad0a3, seq=0x2
6	10.20.0.1	10.10.0.1	ICMP	Echo reply (decoded from ESP)

8. Automatización y CI/CD

Pipeline CI/CD completo con GitHub Actions, 6 etapas de validación.

Eta ­ pa	Herramienta	Función
Lint	Hadolint + ShellCheck	Análisis estático
Security	Trivy (filesystem)	Vulnerabilidades
Build	Docker Buildx	Construcción imagen
Unit Tests	BATS	13 tests
E2E Tests	BATS	15 tests extremo a extremo
Security Scan	Dockle + Trivy	Buenas prácticas + CVEs

Comandos Makefile

```
make build — Construir imagen
make run — Iniciar lab
make status — Verificar SA
make capture — Capturar ESP
make evidence — Evidencias
make test-all — Tests
make lint — Linting
make push — Docker Hub
```

9. Conclusiones

- **Viabilidad:** Entorno completo IPsec IKEv2/ESP con Docker y network namespaces, sin VM.
- **Funcionamiento:** Túnel establecido con AES-CBC-256 / HMAC-SHA-256-128 (RFC 3602, RFC 4868, RFC 7296).
- **Validación triple:** (1) ipsec statusall, (2) XFRM state, (3) decodificación Wireshark/tshark.
- **Automatización:** Makefile con 20+ targets + CI/CD GitHub Actions.
- **Calidad:** ShellCheck, Hadolint, Trivy, Dockle, BATS (28 tests).
- **Reproducibilidad:** Imagen Docker pública + evidencia CLI real.

10. Referencias

- [1] RFC 7296 — C. Kaufman et al., "Internet Key Exchange Version 2 (IKEv2)", 2014
- [2] RFC 4303 — S. Kent, "IP Encapsulating Security Payload (ESP)", 2005
- [3] RFC 3602 — S. Frankel et al., "The AES-CBC Cipher Algorithm and Its Use with IPsec", 2002
- [4] RFC 4868 — S. Kelly & S. Frankel, "Using HMAC-SHA-256 with IPsec", 2007
- [5] RFC 3526 — M. Kojo et al., "More MODP Diffie-Hellman groups", 2003
- [6] strongSwan Documentation — <https://docs.strongswan.org/>
- [7] Docker Network Namespaces — <https://docs.docker.com/engine/network/>
- [8] Wireshark ESP Decryption — <https://wiki.wireshark.org/ESP>
- [9] Linux IPsec (XFRM) — <https://www.kernel.org/doc/html/latest/networking/ipsec.html>
- [10] NIST SP 800-77 — Guide to IPsec VPNs, 2005